Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică
Departamentul Ingineria Software și Automatică

Laboratory Work 6:

Track B: Inside the xv6 File System

Elaborated: gr. FAF-243
Poiata Calin

Verified:
Patricia Reițman

Chișinău – 2026

Table of Contents

Environment & Toolchain 1
Purpose of the Laboratory Work 1
Part 1. Read What xv6 Shows You 1
1. Inspecting xv6 Directory Listings 1
2. Observe Answers for Part 1 2
Part 2. Write `mystat` 3
1. Implementation of `user/mystat.c` 3
2. Watching Hard Links at Work 3
3. Observe Answers for Part 2 4
Part 3. Measure the Limits 4
1. Inspecting Constants and Disk Layout 4
2. Observe Answers for Part 3 5
Part 4. Write `find` 5
1. Implementation of `user/find.c` 5
2. Testing on Fresh Disk 7
3. Stretch Goal: Inode Display via `-n` 7
Conclusion 7

Environment & Toolchain

- **Host Operating System:** Linux (Arch Linux x86_64, Kernel 7.2.9-arch1-1)
- **Host Shell:** GNU bash, version 5.3.20(1)-release (`x86_64-pc-linux-gnu`)
- **Target Operating System:** MIT xv6 RISC-V (`~/xv6-riscv`)
- **Hardware Emulator:** QEMU RISC-V System Emulator (`qemu-system-riscv64` , version 10.2.1)
- **Cross-Compiler Toolchain:** GNU GCC for RISC-V 64-bit (`riscv64-linux-gnu-gcc`), GNU Binutils (`ld` , `objdump` , `nm`)
- **Document Processor:** Typst 0.15.1 (`typst`)

Purpose of the Laboratory Work

1. Understand the structural design and low-level data structures of the xv6 file system, which models modern Unix filesystems (such as ext4) in a compact, readable codebase.
2. Decode xv6 directory listings and inspect file types, inode assignments, and byte sizes directly from `kernel/stat.h` .
3. Implement `user/mystat.c` , a custom user-space utility that invokes the `stat()` system call to inspect file metadata (type, inode number, link count, and byte size).
4. Trace hard link lifecycle mechanics inside the xv6 kernel, specifically inspecting `kernel/sysfile.c` (`sys_unlink`) and understanding when data blocks are deallocated in `kernel/fs.c` (`iput` / `itrunc`).
5. Analyze xv6 filesystem limits and disk geometry defined in `kernel/fs.h` , verifying directory name truncation (`DIRSIZ = 14`), computing maximum file capacity, and auditing the on-disk metadata layout generated by `mkfs` .
6. Implement `user/find.c` , a recursive directory traversal tool based on MIT 6.1810, properly handling entry streams, preventing recursion loops, and closing file descriptors to stay within descriptor boundaries.
7. Implement an optional `-n` stretch flag to inspect inode numbers, demonstrating that hard links share the exact inode of their target while nested regular files receive separate inodes.

Part 1. Read What xv6 Shows You

1. Inspecting xv6 Directory Listings

When xv6 boots, its user shell runs `ls` on the root directory. Unlike POSIX `ls -l` which prints formatted permission strings and ownership names, the minimal xv6 `ls` program outputs raw values extracted directly from `struct stat` :

1. Basename (up to 14 characters)
2. Inode type integer (decoded via `kernel/stat.h`)
3. Inode number on disk
4. File size in bytes

Running `ls` and inspecting newly created directories:

```
$ ls
.              1 1 1024
..             1 1 1024
README         2 2 2441
cat            2 3 35864
echo           2 4 34760
sleep          2 5 34440
```

```
forktest       2 6 17704
grep           2 7 43288
init           2 8 35112
kill           2 9 34664
ln             2 10 34456
ls             2 11 41944
mkdir          2 12 34712
rm             2 13 34696
sh             2 14 57072
stressfs       2 15 35536
usertests      2 16 201520
grind          2 17 50336
wc             2 18 36832
zombie         2 19 34008
logstress      2 20 36680
forphan        2 21 35472
dorphan        2 22 34936
sync           2 23 34112
args           2 24 34424
mystat         2 25 35616
find           2 26 42048
console        3 27 0

$ mkdir a
$ ls a
.              1 28 32
..             1 1 1024
```

2. Observe Answers for Part 1

1. What do types 1, 2 and 3 mean (`kernel/stat.h`)? Inspecting `kernel/stat.h` :

```
#define T_DIR     1   // Directory
#define T_FILE    2   // File
#define T_DEVICE  3   // Device
```

- **Type 1 (`T_DIR`):** Directory file containing an array of 16-byte `struct dirent` records.
- **Type 2 (`T_FILE`):** Regular user data file backed by direct and indirect blocks.
- **Type 3 (`T_DEVICE`):** Special character/block device (e.g., console) mapped to major/minor driver routines rather than persistent disk blocks.

2. Which inode number does the root directory have, and why are `.` and `..` the same there? The root directory has inode number **1** (`. 1 1 1024` and `.. 1 1 1024`). Root is the apex of the filesystem hierarchy and has no parent directory above it. In Unix convention, the root directory's parent pointer `..` self-references the root directory itself, making both `.` and `..` identical pointers to inode 1.

3. An empty directory `a` has size 32. A directory entry has a fixed size. How big is one entry, and which two entries does `a` contain? One directory entry (`struct dirent` in `kernel/fs.h`) consists of:

- `ushort inum` (2 bytes)
- `char name[DIRSIZ]` with `DIRSIZ = 14` (14 bytes)
- Total size per entry: $2 + 14 = \mathbf{16}$ **bytes**.

An empty directory has a size of **32 bytes** because it contains exactly two entries ($2 \times 16 = 32$ bytes):

1. `.` (current directory, pointing to inode 28).
2. `..` (parent directory, pointing to root inode 1).

Part 2. Write `mystat`

1. Implementation of `user/mystat.c`

The `stat()` library call in xv6 populates a user-space `struct stat` defined in `kernel/stat.h`:

```
struct stat {
  int dev;     // File system's disk device
  uint ino;    // Inode number
  short type;  // Type of file
  short nlink; // Number of links to file
  uint64 size; // Size of file in bytes
};
```

We implemented `user/mystat.c` to iterate over path arguments, query metadata via `stat()`, decode the integer type into human-readable text, and output the required formatted string:

```
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"

char *
typename(int t)
{
  if(t == T_DIR)
    return "directory";
  if(t == T_FILE)
    return "file";
  if(t == T_DEVICE)
    return "device";
  return "?";
}

int
main(int argc, char *argv[])
{
  struct stat st;
  if(argc < 2){
    fprintf(2, "usage: mystat <path>...\n");
    exit(1);
  }
  for(int i = 1; i < argc; i++){
    if(stat(argv[i], &st) < 0){
      fprintf(2, "mystat: cannot stat %s\n", argv[i]);
      continue;
    }
    printf("%s: %s, inode %d, links %d, size %d bytes\n",
           argv[i], typename(st.type), st.ino, st.nlink, (int)st.size);
  }
  exit(0);
}
```

2. Watching Hard Links at Work

Executing `mystat` across file operations:

```
$ mystat README . console
README: file, inode 2, links 1, size 2441 bytes
.: directory, inode 1, links 2, size 1024 bytes
console: device, inode 27, links 1, size 0 bytes

$ ln README r2
$ mystat README r2
README: file, inode 2, links 2, size 2441 bytes
r2: file, inode 2, links 2, size 2441 bytes

$ rm README
$ mystat r2
r2: file, inode 2, links 1, size 2441 bytes

$ cat r2 | wc
48 336 2441
```

3. Observe Answers for Part 2

1. Outputs after `ln` and after `rm`. What changed and what did not?

- **What changed:** After `ln README r2`, the hard link counter (`links`) of inode 2 incremented from 1 to 2. Both `README` and `r2` refer to the identical inode 2. After `rm README`, the link counter decremented from 2 back to 1.
- **What did not change:** The inode number (**2**), file type (**file**), byte size (**2441 bytes**), and contents (**48 lines, 336 words, 2441 bytes**) remained unchanged.

2. Find the line in `kernel/sysfile.c` (`sys_unlink`) that decrements the link count. When are the data blocks freed? In `kernel/sysfile.c`, line **244**:

```
ip->nlink--;
iupdate(ip);
```

When data blocks are freed: Data blocks are not deallocated by `sys_unlink`. Data blocks are only returned to the free block pool inside `iput(ip)` in `kernel/fs.c` via `itrunc(ip)` when two conditions are met:

1. `ip->nlink == 0` (no directory entries refer to the inode).
2. `ip->ref == 1` (no open file descriptors hold an active reference to the inode).

3. xv6 has no symbolic links and no permissions. Look at `struct dinode` in `kernel/fs.h`: which fields does an xv6 inode have? In `kernel/fs.h`:

```
struct dinode {
  short type;              // File type
  short major;             // Major device number (T_DEVICE only)
  short minor;             // Minor device number (T_DEVICE only)
  short nlink;             // Number of links to inode in file system
  uint size;               // Size of file (bytes)
  uint addrs[NDIRECT + 1]; // Data block addresses
};
```

- **Fields present:** `type`, `major`, `minor`, `nlink`, `size`, and `addrs[13]` (12 direct + 1 indirect pointer).
- **Fields absent compared to Linux:** No user UID, no group GID, no mode/permission bits, no timestamps (`atime`, `mtime`, `ctime`), and no symlink target buffer.

Part 3. Measure the Limits

1. Inspecting Constants and Disk Layout

```
$ grep -n "define BSIZE\|define NDIRECT\|define NINDIRECT\|define DIRSIZ" kernel/fs.h
5:#define BSIZE   1024 // block size
26:#define NDIRECT   12
27:#define NINDIRECT (BSIZE / sizeof(uint))
54:#define DIRSIZ 14

$ make fs.img 2>&1 | grep nmeta
nmeta 47 (boot, super, log blocks 31, inode blocks 13, bitmap blocks 1) blocks 1953 total
2000

$ mkdir averyveryverylongname
$ ls
.              1 1 1024
..             1 1 1024
averyveryveryl 1 29 32
cat            2 3 35864
```

2. Observe Answers for Part 3

1. What does `ls` show for `averyveryverylongname`, and which constant explains it? `ls` displays `averyveryveryl` (truncated to exactly 14 characters). **Constant:** `#define DIRSIZ 14` in `kernel/fs.h`. Each directory entry allocates exactly 14 bytes for the name buffer (`char name[DIRSIZ]`). Any characters beyond the 14th are dropped by `dirlink()`.

2. Compute the largest file xv6 can hold: 12 direct blocks plus one indirect block that holds block numbers of 4 bytes each, blocks of BSIZE bytes. Show the calculation.

- Block size: `BSIZE = 1024 bytes`
- Pointer size: `sizeof(uint) = 4 bytes`
- Direct blocks: 12 blocks
- Indirect blocks: $\frac{\text{BSIZE}}{\text{sizeof(uint)}} = \frac{1024}{4} = 256$ blocks
- Total addressable blocks: $12 + 256 = \mathbf{268}$ **blocks**
- Maximum file capacity: 268×1024 bytes $= \mathbf{274{,}432}$ **bytes** ≈ 268 KB

3. From the `nmeta` line: how many blocks does the whole disk have, and how many are for the log, the inodes, and the data? From the `mkfs` banner:

```
nmeta 47 (boot, super, log blocks 31, inode blocks 13, bitmap blocks 1) blocks 1953 total 2000
```

- **Total Disk Blocks: 2000 blocks** (`FSSIZE = 2000`).
- **Metadata Blocks (`nmeta`): 47 blocks**:
  - Boot block: **1 block** (block 0)
  - Superblock: **1 block** (block 1)
  - Journal log: **31 blocks** (blocks 2 to 32)
  - Inode blocks: **13 blocks** (blocks 33 to 45, holding $13 \times 16 = 208$ inodes)
  - Free bitmap: **1 block** (block 46)
- **Data Blocks: 1953 blocks** (blocks 47 to 1999).
- Verification: $1 + 1 + 31 + 13 + 1 + 1953 = 2000$ blocks.

Part 4. Write `find`

1. Implementation of `user/find.c`

The `find` program performs recursive directory searches. It reads sequential 16-byte `struct dirent` records from directories, skips unallocated entries (`de.inum == 0`), ignores self-references `.` and `..` to prevent recursion loops, compares basenames with the target string, and recurses into subdirectories. Directory file descriptors are closed immediately upon finishing the iteration to stay within xv6's `NOFILE = 16` process limit.

```c
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"
#include "kernel/fs.h"
#include "kernel/fcntl.h"

void
find(char *path, char *target, int show_inode)
{
  char buf[512], *p;
  int fd;
  struct dirent de;
  struct stat st;
```

```
  if((fd = open(path, O_RDONLY)) < 0){
    fprintf(2, "find: cannot open %s\n", path);
    return;
  }

  if(fstat(fd, &st) < 0){
    fprintf(2, "find: cannot stat %s\n", path);
    close(fd);
    return;
  }

  if(st.type != T_DIR){
    close(fd);
    return;
  }

  if(strlen(path) + 1 + DIRSIZ + 1 > sizeof(buf)){
    fprintf(2, "find: path too long\n");
    close(fd);
    return;
  }

  strcpy(buf, path);
  p = buf + strlen(buf);
  *p++ = '/';

  while(read(fd, &de, sizeof(de)) == sizeof(de)){
    if(de.inum == 0)
      continue;

    char name[DIRSIZ + 1];
    memmove(name, de.name, DIRSIZ);
    name[DIRSIZ] = 0;

    // Skip . and .. to prevent infinite recursion
    if(strcmp(name, ".") == 0 || strcmp(name, "..") == 0)
      continue;

    memmove(p, de.name, DIRSIZ);
    p[DIRSIZ] = 0;

    // Trim trailing padding spaces
    int len = strlen(p);
    while(len > 0 && p[len - 1] == ' ') {
      p[len - 1] = '\0';
      len--;
    }

    if(stat(buf, &st) < 0){
      fprintf(2, "find: cannot stat %s\n", buf);
      continue;
    }

    if(strcmp(name, target) == 0){
      if(show_inode)
        printf("%s %d\n", buf, st.ino);
      else
        printf("%s\n", buf);
    }

    if(st.type == T_DIR){
      find(buf, target, show_inode);
    }
  }

  close(fd);
}

int
main(int argc, char *argv[])
{
  int show_inode = 0;

  if(argc < 3){
    fprintf(2, "usage: find <path> <name> [-n]\n");
    exit(1);
```

```
  }

  if(argc >= 4 && strcmp(argv[3], "-n") == 0)
    show_inode = 1;

  find(argv[1], argv[2], show_inode);
  exit(0);
}
```

2. Testing on Fresh Disk

```
$ mkdir a
$ mkdir a/b
$ echo x > a/b/r2
$ echo y > a/r2
$ ln cat r2
$ find . r2
./a/b/r2
./a/r2
./r2

$ find a b
a/b
```

3. Stretch Goal: Inode Display via `-n`

```
$ find . r2 -n
./a/b/r2 30
./a/r2 31
./r2 3
```

Observe Explanation:

- In `ls`, the `cat` binary has inode number **3** (`cat 2 3 35864`).
- Running `find . r2 -n` reveals that `./r2` has inode **3**. This directly confirms that `./r2` is a hard link sharing inode 3 with `cat`, pointing to the same data blocks without duplication.
- Conversely, `./a/b/r2` (inode 30) and `./a/r2` (inode 31) are distinct regular files created independently with their own allocated inodes and blocks.

Conclusion

Laboratory Work 6 explored the internal mechanisms of the xv6 file system:

1. **Directory Structure:** Demonstrated that an xv6 directory is simply a file containing 16-byte `struct dirent` records. Root's parent `..` loops back to inode 1, and an empty directory starts with 32 bytes representing `.` and `..`.
2. **Metadata Introspection (`mystat`):** Built a custom user-space metadata inspector that prints file type, inode number, link count, and byte size directly from `struct stat`.
3. **Hard Link Lifecycle:** Proved that creating a link increments `nlink` without copying data, and `unlink()` decrements `nlink` while data blocks are only deallocated in `iput()` when `nlink == 0` and no file descriptors remain open.
4. **Limits and Disk Geometry:** Verified directory name truncation at `DIRSIZ = 14`, calculated the maximum file size (274,432 bytes across 12 direct and 256 indirect blocks), and mapped out the 47 metadata blocks versus 1953 data blocks.
5. **Tree Traversal (`find`):** Successfully implemented recursive path walking, properly filtering unallocated records (`de.inum == 0`), preventing recursive loops, and closing descriptors to adhere to kernel limits.